TP — Docker, Docker Compose et Ansible

Oct 14, 2026 · @Elies

Objectifs et environnement

Ce TP part de la commande `docker run` et va jusqu'au déploiement automatisé d'une application sur trois serveurs avec Ansible. Il se déroule en 2 séances de 3 h.

À la fin du TP, vous savez :

- lancer et gérer des conteneurs à partir d'images Docker Hub ;
- décrire une application à plusieurs conteneurs dans un fichier Compose ;
- installer Docker et déployer une stack Compose sur plusieurs hôtes avec Ansible.

Machine	Rôle dans le TP	Système
master01	Poste de travail, nœud de contrôle Ansible, répartiteur de charge (partie 5)	AlmaLinux 10
host01, host02, host03	Hôtes Docker gérés par Ansible	AlmaLinux 10

Prérequis

- Les 4 VMs ont accès à Internet (dépôt Docker et Docker Hub).
- Ansible est installé sur master01 et `ansible docker_hosts -m ping` répond `pong` pour les 3 hôtes. Dans ce document, le groupe d'inventaire s'appelle `docker_hosts` : remplacez-le par le nom de votre groupe.
- Votre utilisateur dispose de `sudo` sur les 4 machines.

Conventions : le prompt `$` indique une commande lancée avec votre utilisateur. Sauf mention contraire, les parties 0 à 2 se font sur master01.

Séance 1 — Partie 0 : installer Docker sur master01 (20 min)

AlmaLinux 10 fournit Podman par défaut. Docker CE s'installe depuis le dépôt officiel de Docker, qui prend en charge les distributions RHEL 10. Vous faites cette installation à la main une fois : en partie 3, Ansible refera exactement ces étapes sur les 3 hôtes.

1. Retirez les paquets qui entrent en conflit avec Docker (ce n'est pas grave s'ils sont absents) :

```
$ sudo dnf remove -y podman buildah runc
```

2. Ajoutez le dépôt Docker CE :

```
$ sudo dnf install -y dnf-plugins-core
$ sudo dnf config-manager --add-repo
https://download.docker.com/linux/rhel/docker-ce.repo
```

3. Installez Docker et le plugin Compose :

```
$ sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-
plugin
```

4. Démarrez le service et activez-le au démarrage :

```
$ sudo systemctl enable --now docker
```

5. Ajoutez votre utilisateur au groupe `docker`, puis **déconnectez-vous et reconnectez-vous** :

```
$ sudo usermod -aG docker $USER
```

6. Vérifiez l'installation :

```
$ docker version
$ docker compose version
$ docker run hello-world
```

Question 0.1 — Pourquoi faut-il se reconnecter après `usermod -aG docker` ? Quel risque de sécurité représente l'appartenance au groupe `docker` ?

Partie 1 : manipuler des conteneurs (1 h)

Toutes les images utilisées sont des images officielles de Docker Hub. Il n'y a rien à construire.

1.1 Un premier serveur web

```
$ docker run -d --name web1 -p 8080:80 nginx
$ docker ps
$ curl http://localhost:8080
$ docker logs web1
```

Entrez dans le conteneur, regardez sa configuration, puis sortez :

```
$ docker exec -it web1 sh
# cat /etc/nginx/conf.d/default.conf
# exit
```

Arrêtez-le, redémarrez-le, puis supprimez-le :

```
$ docker stop web1
$ docker ps -a
$ docker start web1
$ docker rm -f web1
```

Question 1.1 — Que signifient `-d` et `-p 8080:80` ? Quelle différence voyez-vous entre `docker ps` et `docker ps -a` ?

1.2 Un conteneur jetable

```
$ docker run -it --rm alpine sh
/ # cat /etc/os-release
/ # exit
$ docker ps -a
```

Question 1.2 — Le conteneur alpine apparaît-il dans `docker ps -a` ? Quel est le rôle de `--rm` ?

1.3 Servir ses propres fichiers (bind mount et SELinux)

Créez une page web dans votre dossier personnel :

```
$ mkdir -p ~/site
$ echo '<h1>Bonjour depuis master01</h1>' > ~/site/index.html
```

Premier essai, sans option SELinux :

```
$ docker run -d --name web2 -p 8081:80 -v ~/site:/usr/share/nginx/html:ro
nginx
$ curl http://localhost:8081
```

Vous obtenez une erreur **403 Forbidden**. Supprimez le conteneur et relancez-le avec l'option `Z` :

```
$ docker rm -f web2
$ docker run -d --name web2 -p 8081:80 -v ~/site:/usr/share/nginx/html:ro,Z
nginx
$ curl http://localhost:8081
$ ls -Z ~/site
```

Modifiez `~/site/index.html` puis relancez `curl` : la modification est visible immédiatement, sans redémarrer le conteneur.

Question 1.3 — Pourquoi le premier essai échoue-t-il ? Quel contexte SELinux `ls -Z` affiche-t-il après l'option `Z` ?

1.4 Un volume nommé

```
$ docker volume create donnees
$ docker run --rm -v donnees:/data alpine sh -c 'echo "écrit par un
conteneur" > /data/fichier.txt'
$ docker run --rm -v donnees:/data alpine cat /data/fichier.txt
$ docker volume ls
$ docker volume inspect donnees
```

Question 1.4 — Les deux conteneurs ont été supprimés (`--rm`). Pourquoi le fichier existe-t-il encore ? Où est-il stocké sur master01 ?

1.5 Un réseau entre conteneurs

```
$ docker network create labnet
$ docker run -d --name cache --network labnet redis
$ docker run -it --rm --network labnet redis redis-cli -h cache ping
```

La dernière commande doit afficher `PONG`.

Question 1.5 — Comment le second conteneur trouve-t-il l'adresse de `cache` ? Refaites le test sans `--network labnet` : que se passe-t-il ?

1.6 Ménage

```
$ docker rm -f web2 cache
$ docker network rm labnet
$ docker volume rm donnees
$ docker image ls
```

Partie 2 : une application multi-conteneurs avec Compose (1 h 20)

Vous allez faire tourner un WordPress et sa base MariaDB, décrits dans un seul fichier.

2.1 Le fichier Compose

```
$ mkdir ~/wordpress && cd ~/wordpress
```

Créez le fichier `compose.yaml` :

```
services:
  db:
    image: mariadb:11
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: rootpass
      MARIADB_DATABASE: wordpress
      MARIADB_USER: wp
      MARIADB_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    restart: unless-stopped
    depends_on:
      - db
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: wp
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

2.2 Démarrer et observer

```
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f wordpress
```

Quittez les logs avec `Ctrl+C`. Ouvrez `http://master01:8080` depuis un navigateur, terminez l'installation de WordPress et publiez un article.

Interrogez la base depuis le conteneur `db` :

```
$ docker compose exec db mariadb -u wp -pwppass wordpress -e 'SHOW TABLES;'
```

Listez les objets créés par Compose :

```
$ docker network ls
$ docker volume ls
```

Question 2.1 — Comment le conteneur `wordpress` joint-il la base alors qu'aucune adresse IP n'est écrite dans le fichier ? Quel réseau Compose a-t-il créé ?

Question 2.2 — `depends_on` garantit-il que MariaDB est prête à recevoir des connexions quand WordPress démarre ?

2.3 La persistance des données

```
$ docker compose down
$ docker compose up -d
```

Rechargez le site : votre article est toujours là. Recommencez avec l'option `-v` :

```
$ docker compose down -v
$ docker compose up -d
```

Question 2.3 — Qu'a supprimé l'option `-v` ? Dans quel cas l'utiliseriez-vous en production ?

2.4 Sortir les mots de passe du fichier

Créez un fichier `.env` dans `~/wordpress` :

```
DB_ROOT_PASSWORD=rootpass
DB_PASSWORD=wppass
```

Dans `compose.yaml`, remplacez chaque mot de passe écrit en clair par la variable correspondante, par exemple `MARIADB_PASSWORD: ${DB_PASSWORD}`. Vérifiez le résultat, puis relancez la stack :

```
$ docker compose config
$ docker compose down -v && docker compose up -d
```

Question 2.4 — Pourquoi `.env` ne doit-il jamais être publié avec le reste du projet ?

À la fin de la séance, arrêtez la stack avec `docker compose down` pour libérer le port 8080.

Point de contrôle de la séance 1 (20 min)

Répondez par écrit, sans relancer de commande.

1. Quelle est la différence entre une image et un conteneur ?
2. Citez deux façons de rendre des données persistantes et une situation adaptée à chacune.
3. Un conteneur publié avec `-p 8080:80` ne répond pas depuis une autre machine. Citez trois vérifications à faire.
4. Quelle commande Compose équivaut à `docker run` pour tous les services d'un coup ? Et à `docker rm -f` ?
5. Vous lancez deux fois `docker compose up -d` sans rien modifier. Que se passe-t-il la seconde fois ?
6. Pourquoi faut-il préférer une étiquette précise (`mariadb:11`) à `latest` ?

Bilan avant la séance 2 : sur master01, Docker fonctionne et vous savez écrire un `compose.yaml`. Les 3 hôtes, eux, n'ont pas encore Docker.

Séance 2 — Partie 3 : installer Docker sur les 3 hôtes avec Ansible (1 h 15)

Ce playbook reprend une à une les étapes de la partie 0. Toutes les commandes de la séance 2 se lancent depuis master01.

3.1 Préparer le dossier de travail

```
$ mkdir -p ~/tp-ansible/{files,templates} && cd ~/tp-ansible
$ ansible docker_hosts -m ping
```

Le module Compose d'Ansible fait partie de la collection `community.docker`. Vérifiez qu'elle est présente :

```
$ ansible-galaxy collection list community.docker
```

Si la commande ne renvoie rien, installez-la :

```
$ ansible-galaxy collection install community.docker
```

3.2 Le playbook `docker.yml`

```
---
- name: Installer Docker sur les hôtes
  hosts: docker_hosts
  become: true

  tasks:
```

```
    - name: Retirer les paquets en conflit avec Docker
      ansible.builtin.dnf:
        name: [podman, buildah, runc]
        state: absent

    - name: Importer la clé GPG de Docker
      ansible.builtin.rpm_key:
        key: https://download.docker.com/linux/rhel/gpg
        state: present

    - name: Ajouter le dépôt Docker CE
      ansible.builtin.get_url:
        url: https://download.docker.com/linux/rhel/docker-ce.repo
        dest: /etc/yum.repos.d/docker-ce.repo
        mode: "0644"

    - name: Installer Docker et le plugin Compose
      ansible.builtin.dnf:
        name: [docker-ce, docker-ce-cli, containerd.io, docker-compose-
plugin]
        state: present

    - name: Démarrer et activer Docker
      ansible.builtin.systemd_service:
        name: docker
        state: started
        enabled: true
```

Lancez-le, puis relancez-le une seconde fois :

```
$ ansible-playbook docker.yml
$ ansible-playbook docker.yml
```

Question 3.1 — Comparez les compteurs `changed` du récapitulatif (`PLAY RECAP`) entre les deux exécutions. Comment s'appelle cette propriété et pourquoi est-elle importante ?

3.3 Vérifier à distance

```
$ ansible docker_hosts -a "docker --version"
$ ansible docker_hosts -b -a "docker run --rm hello-world"
```

Question 3.2 — La seconde commande utilise le module `command` par défaut. Pourquoi faut-il `-b` ici ?

3.4 Ajouter une configuration et un handler

Par défaut, les journaux des conteneurs grossissent sans limite. Créez `files/daemon.json` :

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

Ajoutez ces deux tâches à la fin de `tasks`, ainsi qu'une section `handlers` au même niveau que `tasks` :

```
    - name: Créer le dossier de configuration de Docker
      ansible.builtin.file:
        path: /etc/docker
        state: directory
        mode: "0755"

    - name: Limiter la taille des journaux des conteneurs
      ansible.builtin.copy:
        src: files/daemon.json
        dest: /etc/docker/daemon.json
        mode: "0644"
      notify: Redémarrer Docker

  handlers:
    - name: Redémarrer Docker
      ansible.builtin.systemd_service:
        name: docker
        state: restarted
```

Lancez le playbook deux fois de suite.

Question 3.3 — Lors de quelle exécution le handler s'est-il déclenché ? Pourquoi ne pas simplement mettre une tâche « redémarrer Docker » à la fin du playbook ?

Partie 4 : déployer une stack Compose sur les 3 hôtes (1 h)

Chaque hôte va servir une page nginx qui affiche son propre nom. Le fichier Compose est le même partout. Seule la page change : un template Ansible la génère pour chaque hôte.

4.1 Les fichiers à déployer

`files/compose.yaml` :

```
services:
  web:
    image: nginx:latest
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./site:/usr/share/nginx/html:ro
```

`templates/index.html.j2` :

```
<!doctype html>
<html>
  <body style="font-family: sans-serif">
    <h1>Servi par {{ inventory_hostname }}</h1>
    <p>Adresse : {{ ansible_default_ipv4.address }}</p>
  </body>
</html>
```

4.2 Le playbook `app.yml`

```
---
- name: Déployer le site web sur les hôtes
  hosts: docker_hosts
  become: true

  tasks:
    - name: Créer le dossier du site
      ansible.builtin.file:
        path: /opt/web/site
        state: directory
        mode: "0755"
        setype: container_file_t

    - name: Copier le fichier Compose
      ansible.builtin.copy:
        src: files/compose.yaml
        dest: /opt/web/compose.yaml
        mode: "0644"

    - name: Générer la page d'accueil
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: /opt/web/site/index.html
        mode: "0644"
```

```
        setype: container_file_t

    - name: Démarrer la stack
      community.docker.docker_compose_v2:
        project_src: /opt/web
        state: present
```

Ici, le `Z` de la partie 1 n'est plus nécessaire : c'est Ansible qui pose le contexte SELinux `container_file_t` avec l'option `setype`.

```
$ ansible-playbook app.yml
$ curl http://host01
$ curl http://host02
$ curl http://host03
```

Chaque hôte doit renvoyer une page avec son propre nom.

Question 4.1 — D'où viennent `inventory_hostname` et `ansible_default_ipv4.address` ? Lequel des deux nécessite la collecte des faits (`gather_facts`) ?

Question 4.2 — Que signifie `state: present` pour le module `docker_compose_v2` ? À quelle commande Compose correspond-il ?

4.3 Modifier le contenu

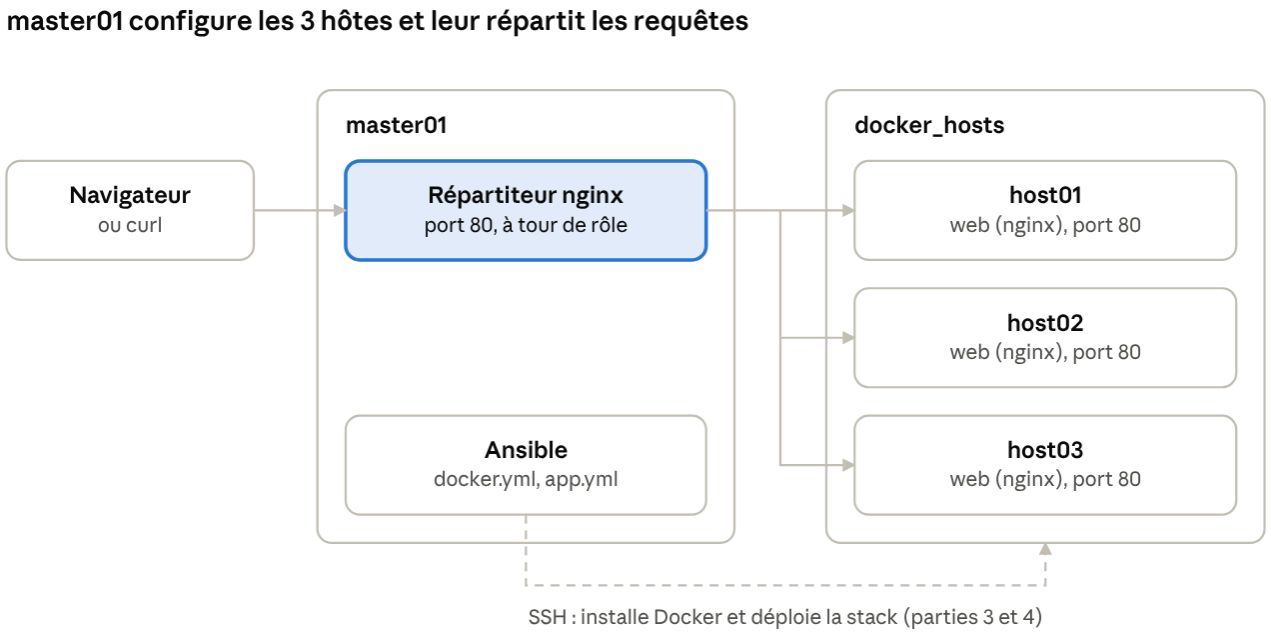
Ajoutez une ligne `<p>Version 2</p>` dans le template, puis relancez le playbook.

Question 4.3 — Quelles tâches sont `changed` ? La nouvelle page est servie sans redémarrer le conteneur : pourquoi ?

Question 4.4 — Arrêtez la stack de host02 avec `ansible host02 -b -a "docker compose -f /opt/web/compose.yaml down"`, puis relancez `app.yml`. Sur quel hôte la dernière tâche est-elle `changed` ?

Partie 5 : répartir la charge depuis master01 (45 min)

Un nginx lancé avec Compose sur master01 reçoit les requêtes et les distribue à tour de rôle aux 3 hôtes.



architecture finale · master01 et 3 hôtes

Les flèches pleines montrent le trafic web, et la flèche en pointillés le pilotage par Ansible en SSH.

5.1 Relever les adresses des hôtes

Le conteneur ne lit pas le fichier `/etc/hosts` de master01. Il faut donc écrire les adresses IP dans la configuration. Relevez-les :

```
$ getent hosts host01 host02 host03
```

5.2 La configuration nginx

```
$ mkdir ~/lb && cd ~/lb
```

Créez `nginx.conf` en remplaçant `IP_HOST01`, `IP_HOST02` et `IP_HOST03` par les adresses relevées :

```
upstream web {
    server IP_HOST01:80;
    server IP_HOST02:80;
    server IP_HOST03:80;
}

server {
    listen 80;
    location / {
        proxy_pass http://web;
    }
}
```

Créez `compose.yaml` :

```
services:
  lb:
    image: nginx:latest
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro,Z
```

5.3 Tester la répartition

```
$ docker compose up -d
$ for i in 1 2 3 4 5 6; do curl -s http://localhost | grep Servi; done
```

Les noms host01, host02 et host03 doivent apparaître à tour de rôle.

Question 5.1 — Quel algorithme de répartition nginx utilise-t-il par défaut ?

5.4 Simuler une panne

```
$ ansible host02 -b -a "docker compose -f /opt/web/compose.yaml down"
$ for i in 1 2 3 4 5 6; do curl -s http://localhost | grep Servi; done
```

Question 5.2 — Le site reste-t-il disponible ? Quels hôtes répondent ?

Remettez host02 en service avec Ansible :

```
$ ansible-playbook ~/tp-ansible/app.yml
```

5.5 Bonus : générer `nginx.conf` avec Ansible

Ajouter un 4e hôte oblige aujourd'hui à modifier `nginx.conf` à la main. Écrivez un template `lb.conf.j2` qui construit la liste `upstream` à partir du groupe d'inventaire :

```
upstream web {
{% for h in groups['docker_hosts'] %}
    server {{ hostvars[h].ansible_default_ipv4.address }}:80;
{% endfor %}
}
```

Le bloc `server` reste identique à celui de la partie 5.2. Écrivez ensuite un play qui cible `master01` (ajoutez-le à l'inventaire s'il n'y figure pas). Ce play génère `~/lb/nginx.conf` à partir du template, puis relance la stack du répartiteur avec `docker_compose_v2` et `state: restarted`.

Question 5.3 — Pourquoi le play qui génère ce template doit-il venir après un play qui a collecté les faits des 3 hôtes ?

Nettoyage et barème

Nettoyage en fin de TP

```
$ cd ~/lb && docker compose down
$ ansible docker_hosts -b -a "docker compose -f /opt/web/compose.yaml down"
$ cd ~/wordpress && docker compose down -v
```

Barème (sur 20)

Chaque partie est notée sur la vérification en direct du résultat et sur les réponses aux questions qui s'y rapportent.

Partie	Ce qui est vérifié	Points
0. Docker sur master01	`docker run hello-world` fonctionne sans `sudo` · question 0.1	1
1. Conteneurs	Page servie en bind mount avec `Z`, volume persistant, `PONG` sur le réseau `labnet` · questions 1.1 à 1.5	4
2. Compose	WordPress opérationnel, article conservé après `down` / `up`, mots de passe dans `.env` · questions 2.1 à 2.4	4

Partie	Ce qui est vérifié	Points
Point de contrôle	Réponses écrites aux 6 questions	2
3. `docker.yml`	Docker sur les 3 hôtes, seconde exécution à `changed=0`, handler déclenché une seule fois · questions 3.1 à 3.3	3
4. `app.yml`	3 pages différentes servies, modification du template déployée · questions 4.1 à 4.4	4
5. Répartiteur	Rotation sur les 3 hôtes, service maintenu pendant la panne de host02 · questions 5.1 et 5.2	2
Bonus 5.5	`nginx.conf` généré par Ansible à partir de l'inventaire	+1